

Final Report

ChainProof - Monark

Team # 6

A Report
Presented to
The Department of Electrical & Computer Engineering
Concordia University

In Partial Fulfillment
of the Requirements
of ELEC/COEN 490

by

Remi C. Jubinville	ID: 40228517
Ireoluwa Onamusi	ID: 40185045
Najmuddin Nazary	ID: 40211776
Daran Guo	ID: 40127349
William Ma	ID: 40215391
Fady Dous	ID: 40190778

Project Supervisor
Dr. Yan Liu

Concordia University
April 2026

Table of Contents

Abstract	4
1. Introduction	5
1.1 Objectives and Outline	5
Project Objectives	5
Report Outline	6
1.2 The “Sales Pitch”	6
2. Final Product Specifications	7
2.1 System Requirements	7
2.2 Hardware Specifications	7
2.3 Software Specifications	8
3. System Description and Design	9
3.1 Hardware Design	9
3.2 System Level Software Design	10
3.3 Data Flow & Logic	10
3.4 Smart Contracts Logic	13
4. Implementation	14
4.1 Development Process	14
Phase 1: Project Initiation and Resource Allocation	14
Phase 2: System Design and Scope Adjustment	14
Phase 3: Core Implementation, Prototyping, and Architectural Pivots	15
Phase 4: Final Refinement and Hardware Iteration	15
Testing Methodology	16
4.2 Schedule Deviations	17
5. Results and Assessment	17
5.1 Test Conditions	17
5.2 Hardware and Sensor Testing	18
5.3 System Integration and Data Transfer	19
5.4 Analysis of Results	19
6. Conclusion and Lessons Learned	20
6.1 Final Conclusions	20
Project Success and Assessment	20
Hardware and Interaction Improvements	20
Architectural and Network Expansions	21
Ecosystem and Security Integration	21
6.2 Feedback on Lessons Learned	22

Navigating Dual-Stakeholder Requirements	22
Proactive Scope Validation	22
Team Composition and Technical Adaptability	22
Perseverance in Prototyping	23
References	24
Appendix A: The second ELSEE assignment.	25
Part A: 8-Factor ELSEE Considerations	25
PART B: Linked Cross-References From Matrix	28
PART C: Stakeholders	30
PART D: Consideration of Risk/Failure & Remedies in Our Capstone Domain	31
PART E: Possible Venues for Publication of Our Findings	32
Appendix B: Technical Manual	33
1. Introduction	33
2. Objective	33
3. Product Specifications	34
4. System Block Diagram	35
5. Description of Operation	36
6. Hardware description	36
6.1 Main Hardware Components	36
6.2 Hardware Interconnection Summary	37
6.3 Hardware Functional Description	38
7. Software and Firmware Description	38
7.1 Firmware Description	38
7.2 Firmware Flow Chart	40
7.3 Web Application Description	40
7.4 Wallet Authentication Description	41
7.5 Smart Contract Description	42
7.6 Role-Specific Logic	42
8. Test and Debugging	43
9. Technical Limitations and Recommended Improvements	44
Appendix C: User Manual	44
Purpose	44
Who Should Use This Manual	44
Before You Start	45
Important Usage Notes	45
Main Pages	46
Sign In and Connect Your Wallet	46
Apple Device Note	46
Assign Your Role	47

Start an NFC-Based Workflow	47
Producer Workflow	48
Create a Batch and Bind Hardware	48
Transfer the Batch	48
Warehouse Workflow	49
Receive a Batch	49
Transfer a Batch Onward	49
Transporter Workflow	50
Receive a Shipment	50
Deliver a Shipment	50
Track a Batch	50
Note About Tracked Batches	51
Common Problems and What To Do	51
Wrong Network	51
No Role Assigned	51
No Batch Found for Hardware	52
No Scan Context Available	52
Wallet Transaction Rejected	52
Recipient Address Problems	53
What the Platform Verifies	53
Good Operating Practice	54

Abstract

De-centralized supply chains often have difficulty in maintaining transparency, integrity, and security for record keeping of the items that are in transit, especially related to the environmental conditions of the items during transit. This study provides a description of the concept, development, and testing process for ChainProof which is a blockchain-based system that was designed specifically to overcome this challenge. The prototype is based on a PoC concept for shipping coffee beans and utilizes a combination of IoT edge devices and blockchain technology. The hardware prototype developed using the soldered protoboard consists of an ESP32 microcontroller, a DHT22 environmental sensor, and an ST25DV16K NFC module that facilitate constant logging of temperatures and humidity data while providing convenient "tap-to-read" interaction with a user's smartphone. As a way of minimizing expenses and maximizing security, ChainProof adopts a simple software architecture with all important state changes and threshold breaches being executed by Ethereum smart contracts. Additionally, the use of a decentralized identity management system such as MetaMask guarantees secure role-based access control and immutable proof of origin. Testing of the solution confirmed its capability of logging telemetry information, transferring data with the help of an NFC tag, as well as automatic cancellation of any shipments in case of environmental breaches. In spite of going through several iterations in terms of design decisions and functionality, the developed prototype serves as sufficient validation for the concept of blockchain-based IoT telemetry application.

1. Introduction

1.1 Objectives and Outline

Project Objectives

ChainProof Project Description The key problem tackled by ChainProof is the one regarding secure and transparent recording of transport of goods through the decentralized chain of supply. The main goal is to establish a blockchain-based tracing process ensuring that all involved parties can trust each other. Although the system has a modular design, its particular use is aimed at addressing the process of shipping coffee beans.

For this purpose, the system operates based on the following primary mechanisms:

- **Immutability of the Custody Chain:** The participants include producers, carriers, processors, warehouses, and customers who keep logs of all their actions to document the transfer of custody.
- **Smart Contracts:** With the use of a blockchain ledger and smart contracts, the system ensures that none of the information can be altered, manipulated, or deleted from its historical records.
- **Seamless Integration with Hardware:** In order to make it possible for participants to interface with the system, NFC (Near Field Communication) technology is incorporated into it so that participants can interact with the system by tapping the NFC chip embedded in their phones.

- **Environmental Sensing:** A specialized IoT sensor, which is placed on the pallets, continuously monitors the temperature and humidity of the coffee beans during transportation.
- **Excursion Detection:** When the temperature and humidity exceed the predefined threshold, an automated mechanism will detect the excursions and classify the product as non-conforming, permanently invalidating its history.

Report Outline

The rest of this report will cover the entire lifecycle of the development process of the ChainProof system. Section 2 explains the product specifications for the end-product. Section 3 covers the architecture and design of the system in detail. This section will also explain the technical decisions taken at various points in time during the design process. Section 4 talks about the different phases of implementation and discusses the deviations from the schedule. Lastly, Section 5 provides details on testing along with its results.

1.2 The “Sales Pitch”

One of the main difficulties associated with decentralized supply chains in any industry is the problem of secure documentation of the transport process, and tracking environmental conditions during this process transparently. ChainProof delivers the solution to this problem, it is a proof of concept for creating a cheap, scalable, and verifiable chain from production to delivery of goods.

The key advantages of ChainProof can be summarized into three main points. First of all, it uses an efficient condition monitoring approach. To make sure that the quality standards for goods are

met, the company developed a special low-power IoT gadget that monitors the environment inside the transport vehicle and collects information about critical parameters such as temperature and humidity. Secondly, ChainProof offers enhanced data security through a smart approach to dealing with gas fees related to working with Ethereum^[8]. Thanks to the lean architecture and fully on-chain approach to storing data, ChainProof stores transactions and environmental data only when they are needed to be stored on a decentralized ledger to ensure that nothing changes in the future. Thirdly, ChainProof uses decentralized authentication through MetaMask. It means that every operation performed by users is cryptographically signed, ensuring strict role-based access control.

2. Final Product Specifications

2.1 System Requirements

- **Immutable Ledger:** All transactions and transfers, along with all verifications, should be logged safely.
- **Role Based Access Control (RBAC):** Access and writing permissions should be separated based on the role of the stakeholder involved; Producer, Transporter, Processor, Warehouse, and Consumer.
- **Automated Smart Contracts:** Important processes such as delivery and authenticity verification should be automated through smart contracts without human intervention.

2.2 Hardware Specifications

- **Architecture Prototype:** The actual tracking hardware is realized in a PoC setup using a tethered design. Every component and trace was connected to the protoboard through physical soldering, which proved the ability of the hardware without having to design a PCB or power management module.
- **Microcontroller:** The core of the entire system is an ESP32-DEVKIT module^[4], powered through a USB link from the user's laptop computer.
- **NFC Interface:** An ST25DV16K NFC module^[3] is embedded in the protoboard to facilitate "tap-and-go" operations of the device using the user's smartphone.
- **Environmental Monitoring:** The temperature and humidity level of the surroundings are continuously monitored by a DHT22 module^[5].

2.3 Software Specifications

- **System Architecture:** An optimized on-chain design architecture that stores the essential state transitions, events, and light weights in the Ethereum blockchain ecosystem^[8]. The system helps in reducing gas costs and avoiding conflicts while synchronizing data due to the decentralized nature of the blockchain.
- **Frontend Framework:** The frontend framework used is based on the responsive web app framework, which uses Next.js^[7] to perform server-side rendering for handling role-based dashboards.
- **Authentication:** Authentication is done using the decentralized identity solution through MetaMask, and all the actions are cryptographically signed with private keys saved only on the local devices of the users.

3. System Description and Design

3.1 Hardware Design

The ChainProof Physical Tracking Layer is designed as a tethered Proof of Concept (PoC). In order to maintain proper electrical connectivity without using a customized printed circuit board, the entire assembly was mounted on a protoboard. Electrical diagram schematics of the prototype are shown in Figure 1 below. The central processing device used in this device is ESP32-DEVKIT^[4] and the two peripheral devices connected are the DHT22 Sensor^[5] for temperature and humidity readings and the ST25DV16K NFC chip^[3] for communication with smartphones. Since there is no provision for power management or a Battery Management System (BMS) in this prototype, power is supplied by connecting the microcontroller to a USB port on a laptop computer.

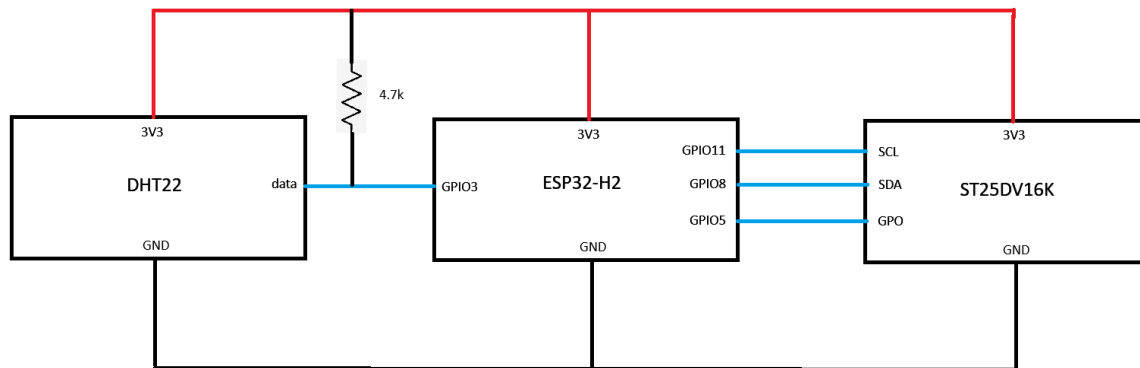


Figure 1: Electrical Wiring Schematic of the Tethered Prototype

3.2 System Level Software Design

The ChainProof suite leverages its efficiency and reliability by adopting a simple and completely on-chain architecture. See figure 2 for an overview of the overall end-to-end data flow within the entire infrastructure framework. The design of the system consciously avoids the need for any off-chain data storage mechanisms to avoid any synchronization challenges since there is only one source of truth, namely the decentralized ledger. The system takes advantage of a sequential data flow where interactions from the edge are captured via the NFC interface and responded to by the Next.js^[7] frontend that sends out data to the Express API backend.

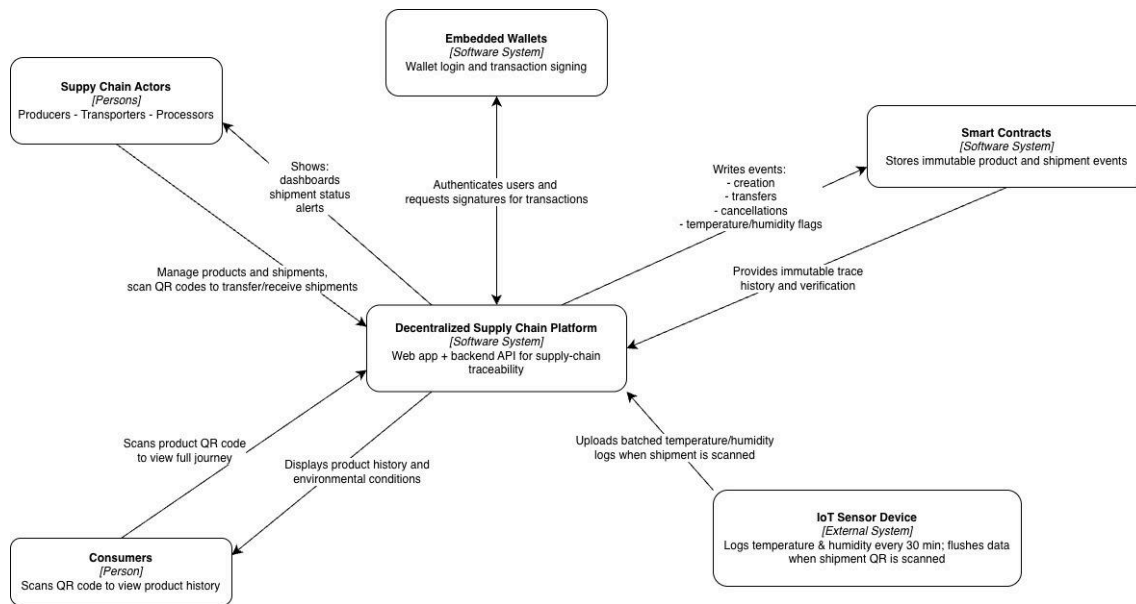


Figure 2: High-Level End-to-End System Architecture

3.3 Data Flow & Logic

The main interaction that happens in the ChainProof system is called the "tap-to-read" function, which links physical logistics to digital traceability. The flow of operations for the tap-to-read process is illustrated in Figure 3 below. The flow starts when the user taps his phone against the ST25DV16K NFC chip^[3] and sends the shipment ID and telemetry securely to the frontend

based on Next.js^[7] framework. Then, the user signs the transaction using the MetaMask wallet. Following the completion of the signature phase, the transaction is verified by the Express server to generate a Merkle leaf hash. Finally, the generated Merkle hash is sent to the Ethereum blockchain^{[6][8]} network for verification by the smart contract before being sent back to the user's dashboard.

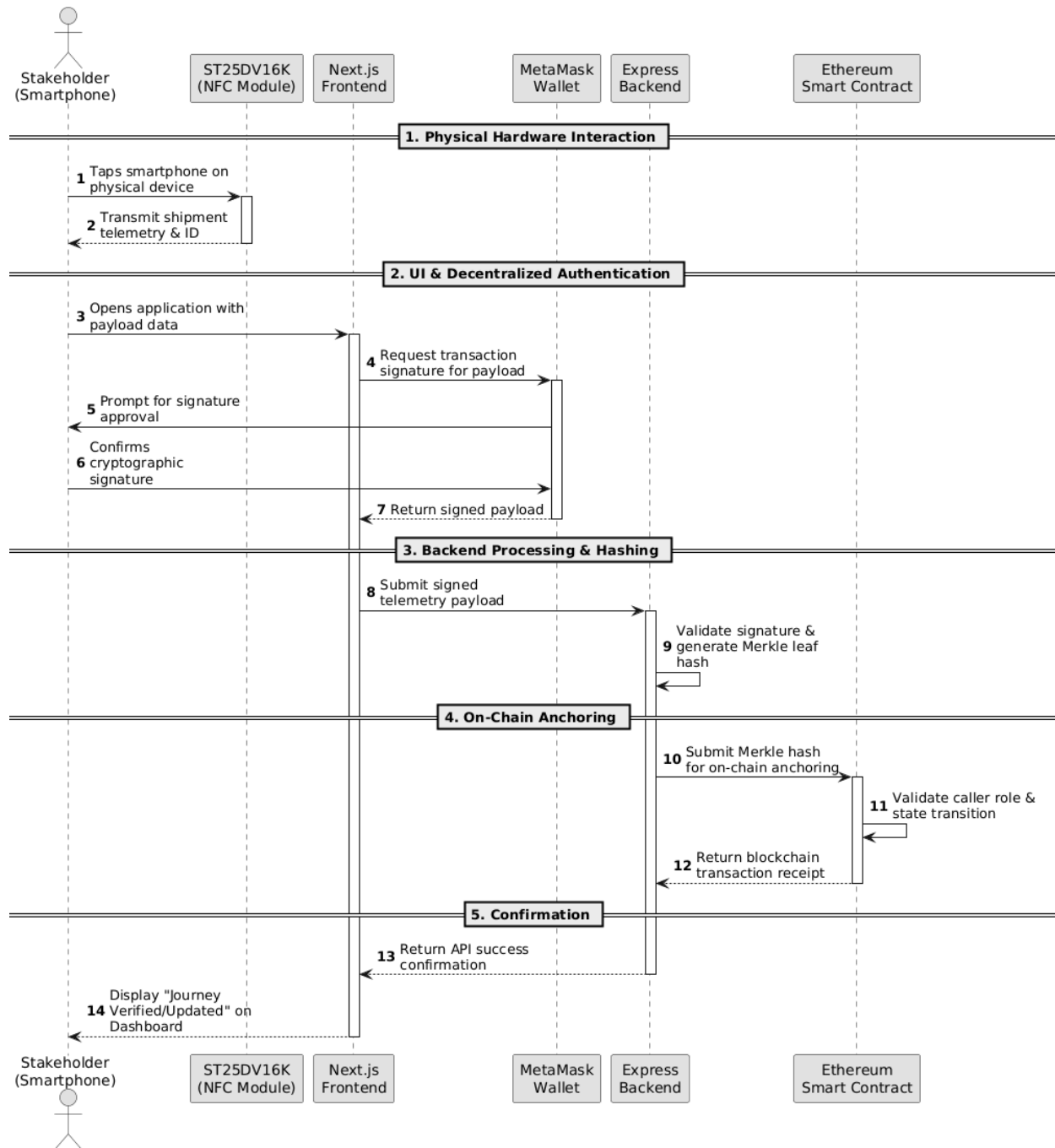


Figure 3: Sequence Diagram of the "Tap-to-Read" Interaction and On-Chain Anchoring

3.4 Smart Contracts Logic

The integrity of the supply chain is guaranteed solely by the business logic that is programmed into the Ethereum smart contracts^{[6][8]}. The state machine responsible for regulating the life cycle of the coffee shipment is presented in Fig. 4 below. It can be observed that there is an orderly transition between states. The shipment starts out with the status of "Created" by the Producer and moves on to "In Transit," facilitated by the Transporter. Finally, it ends up being "Received" and "Processed" by the relevant parties at the destination. More importantly, the smart contract features automation of the excursion condition. Whenever the environmental telemetry data, such as temperature and humidity, falls outside of acceptable limits set at the time of submission, the smart contract automatically changes the shipment's status to "NonCompliant," which is an irreversible final stage.

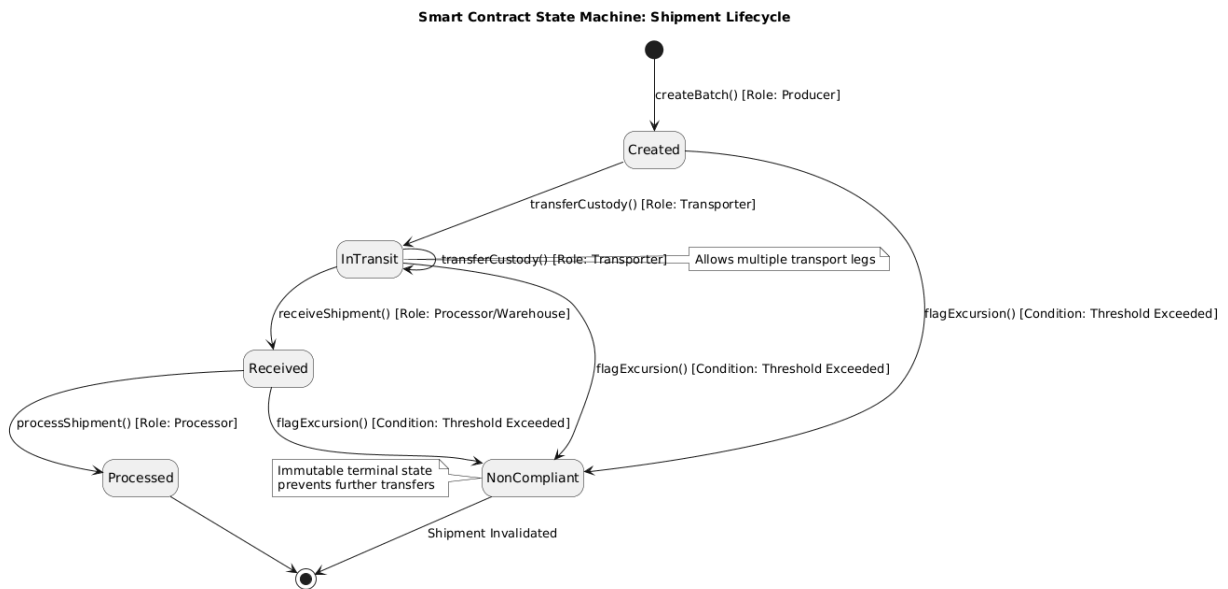


Figure 4: State Machine Diagram of the Shipment Lifecycle

4. Implementation

4.1 Development Process

Phase 1: Project Initiation and Resource Allocation

The project began with the choice of the ChainProof framework, suggested by our industry client, Monark^[1]. As the software architecture and blockchain technology formed the main focus of the project, we deliberately created a team that would favor software engineering (five Computer Engineering and one Electrical Engineering major). The versioning and collaboration of the project would be facilitated by Monark's GitHub account. Our communication lines would be established in advance. Client communications would be conducted via Monark's Discord server and bi-weekly Google Meets scrums (every other Friday at 10:00 AM). In-house coordination of the team would be conducted via Discord servers (text-based and voice-based once per week on Mondays at 1:00 PM during Phases 1 and 2). Initial division of duties was conducted based on the competencies of the team members involved.

Phase 2: System Design and Scope Adjustment

In the process of designing, the team managed to build the whole software architecture from the blockchain level up through smart contracts and other high-level system functions. It was near the end of Phase 2 that an important shift in the scope happened. As it turned out, there was a need for a mandatory academic deliverable, which is a hardware device (in particular, a printed circuit board). While it did not seem necessary at first, considering the software-oriented scope, team members who were working on the backend had no choice but to turn to hardware work.

Phase 3: Core Implementation, Prototyping, and Architectural Pivots

After the holidays, the implementation of the software was fast-tracked. The smart contracts, user interface (UI), and back-end implementation were completed successfully. In this stage, the internal team meetings were rescheduled to Tuesday afternoon at 1:30 p.m.

An important change that took place after consultations with the client about the off-chain database involved focusing on the main functionalities required by the demos and deciding to forego the database layer, which was supposed to store metadata such as organizations and product types. In order to prevent synchronization problems, it was decided that all state information, including users, orders, and batch IDs, needed to be stored on-chain only. As a result, the implementation of the database was postponed.

Meanwhile, the hardware team had trouble receiving prompt feedback from technical advisors, which complicated the process of designing the custom PCB. To produce a functional product for testing purposes ahead of the actual demo, the team built a tethered PoC using an ESP32-DEVKIT^[4] and a DHT22 sensor^[5] connected to the breadboard. Initially, it was planned to use an Adafruit PN532 NFC controller; however, owing to driver and compatibility problems, it was decided to temporarily switch to Bluetooth (BLE) connectivity with a local host computer. Despite some performance issues when connected via BLE, the system succeeded in demonstrating the key functionality, namely, data transfer and automated shipment invalidation according to environmental thresholds.

Phase 4: Final Refinement and Hardware Iteration

Since the software architecture was mostly in place, Phase 4 saw efforts being put into improvements in the user interface design, such as visualizing hardware data transfers and

completing the blockchain layer. The database layer is postponed at this point and will potentially be implemented following Phase 4 if there is an interest from the client side. The main technical difficulty was associated with the physical hardware implementation. It turned out that Adafruit PN532 chosen in the beginning was not capable of doing peer-to-peer smartphone data transfers, therefore an optimized Bluetooth Low Energy connection had to be used for the initial in-lab demonstration. But before the final presentation of the work, the team managed to build the desired hardware, which resulted in the successful completion of the project goal. The final Proof of Concept prototype was constructed and soldered together using a protoboard and an ST25DV16K NFC chip^[3].

Testing Methodology

During the development process, there was greater emphasis on manual integration testing as well as end-to-end functional testing rather than automated unit testing suites. Regarding the blockchain layer, the smart contract code was first tested via local development environments (for example, Hardhat) to ensure that state changes and the role access control mechanism were working properly before the actual implementation. As for the frontend side, the UI was validated manually through simulations of different user journeys to ensure that MetaMask authentication and role-based routing as well as real-time dashboards work properly. The hardware streams of data, finally, were tested physically during the process of building the prototype board PoC. This was accomplished by physically initiating an interaction with the ST25DV16K device and observing the propagation of data from the edge device through the frontend application to the decentralized ledger.

4.2 Schedule Deviations

There were several instances of deviation from the original schedule presented during Phases 1 and 2 of the project due to delays caused by changes in the scope. The original schedule for both phases was formulated based on an output that only consisted of software, such as the blockchain ledger, backend, and Next.js frontend.

The most significant delay came up towards the latter part of Phase 2 when the academic committee made it known that the development of custom physical hardware would be one of the prerequisites for passing the course. This resulted in a need for reallocating engineering manpower to a task which was not initially planned, causing the missed deadline of the Design Approval set out for November 22nd. Due to such communication barriers between backend developers and technical advisors when moving on to Phase 3, further delay ensued in the phase schedule due to the need for design approval. It eventually led to the failure of developing a custom PCB in Phase 3. The project had no choice but to make major adjustments to its Phase 4 schedule in order to facilitate the development of the tethered protoboard PoC in an iterative manner. This pushed the final integration testing of the ST25DV16K NFC module^[3] to the last stages of the project lifecycle.

5. Results and Assessment

5.1 Test Conditions

The testing was performed in two different environments in order to ensure validation not only of the software architecture but also of the actual edge devices. The smart contracts, database layer

(prior to the deprecation of this layer), and API endpoints were first validated using synthetic datasets. In particular, a specialized sensor generator script was used to create both regular environmental states and excursion data (for example, in the form of sudden temperature or humidity peaks). Such an approach made it possible to test backend logic, generate Merkle roots, and anchor them to the blockchain under varying loads, not requiring any physical device to do so. The actual testing was then performed as part of the final in-lab demo. The physical testing setup comprised the PoC hardware connected to the perf-board in a tethered configuration. The ESP32-DEVKIT board^[4] was kept powered through a USB connection to the host laptop. The data transfer to the frontend and backend was verified in a localized environment through BLE.

5.2 Hardware and Sensor Testing

The foremost purpose of the edge device for the hardware component was to capture and transmit environmental telemetry efficiently. Since the final implementation of the prototype was a tethered PoC, it was never necessary to test it for autonomous power capabilities such as battery backup and protection against discharging. The device depended entirely on host power, thereby achieving 100% uptime at all times within the timeframe of the tethered demo. In terms of capturing environmental data, the DHT22^[5] was effective in acquiring ambient data pertaining to temperature and humidity. Environmental inputs were purposely made outside of the safety threshold values defined by the stakeholders. The inputs were correctly identified as excursions, with the batch flagged for shipment.

5.3 System Integration and Data Transfer

Data pipeline validation between the edge device and the blockchain was another important stage in the development process. Even though there were some difficulties with the initial protocol choice of integrating Adafruit PN532 NFC Controller, the shift in strategy towards BLE turned out to be quite effective. The telemetry messages have been sent reliably to the local host without any considerable loss of packets throughout the demonstration period. What is more, the hybrid data model functioned perfectly fine in both test and live environments. All incoming sensor logs have been processed by the backend API, and the required Merkle leaf hashes have been produced and the write operations on the smart contracts completed. Therefore, the journey of the product has been accurately reflected on the frontend page, which clearly showed that the meta-mask authenticated access control mechanism worked effectively.

5.4 Analysis of Results

The final evaluation of the PoC concludes that the project was very successful in meeting the key goals of the software and traceability aspect. Transactions are recorded securely; the access control process is robust; and there is an immutable record of how the item passed through the various stages in the supply chain. The successful PoC implementation of automated shipment rejection demonstrates that it is possible to combine physical world IoT telemetry and blockchain technology. Although the hardware was not exactly as expected initially and had dependencies in terms of power and BLE connectivity, it is well architected for the next phase in its evolution with custom hardware support.

6. Conclusion and Lessons Learned

6.1 Final Conclusions

Project Success and Assessment

The ChainProof development project was completed successfully and resulted in a working Proof of Concept, which satisfied all software and traceability-related requirements put forward by Monark^[1] – the industry client. It was evident from the implementation that a decentralized approach could ensure complete visibility throughout the supply chain as well as secure and cost-effective tracking of environmental conditions and custody of items. Although the project encountered difficulties during hardware design because of certain environmental factors, the software framework itself, from the frontend based on Next.js to the Ethereum smart contracts, was solid and sufficient.

Hardware and Interaction Improvements

As for future versions, the first step will be converting the tethered version into a PCB with an integrated low-power microcontroller, necessary voltage regulators, and a BMS designed for LiPo batteries with a stable connection port. Logistical realities call for an increase in energy efficiency such that the hardware has at least three months of independent use. It also needs an independent, power-stable storage medium where it will be able to store sensor data for three months straight in order to avoid any possible loss while in transit. With regards to user interactions, the NFC protocol used needs to be improved significantly in order to create a smooth “tap-to-read” functionality and optimize the MetaMask authentication process.

Architectural and Network Expansions

The high modularity of the ChainProof system design makes it infinitely scalable. In conversations with the project supervisor, Dr. Liu, regarding the system's architecture, one very feasible future upgrade would be the implementation of a localized hierarchical structure or mesh network for each shipment. Rather than considering each pallet as its own communication node, a large "master" node containing a GPS tracker could be attached to the transport truck itself. The smaller IoT nodes in each individual pallet would act as a localized network under this master node. With this architecture in place, a transporter would simply have to tap the master node to automatically produce batch IDs and record any transfer events from all subordinate nodes.

Ecosystem and Security Integration

Finally, in order for the platform to mature into its commercial form, it will have to be embedded within a much bigger and more comprehensive delivery management system that handles the assignment of delivery personnel and facilitates automatic custody exchanges through geofencing and proximity algorithms via GPS technology. Lastly, in order to further strengthen the trust model of the system, it is highly advised that future versions of the solution incorporate a rigorous Know Your Customer (KYC) algorithm in which a governmentally approved identity will automatically be linked with a user's MetaMask wallet.

6.2 Feedback on Lessons Learned

Navigating Dual-Stakeholder Requirements

Working together with Monark^[1] gave us first-hand experience with software development and dealing with clients. The biggest lesson learned in working with clients was that the client should be satisfied, but communicating technical information to the clients who do not have much knowledge on the subject can be difficult. Moreover, what was hard to deal with was the conflict we had to face when we were placed in between two parties which were measuring their achievements using two completely different sets of standards. Our experience in trying to meet our clients' demands solely based on software while the university required physical engineering made our work suffer.

Proactive Scope Validation

An important lesson learned in the area of project management was the price paid for a delay in technical consultations. At the beginning of the process, the team assumed that this project was purely related to software development without paying attention to the physical design of the hardware in the initial two phases. The reason why the team failed to obtain early confirmation on the design from the academic evaluator was that it came to realize late the importance of the deliverable that needed to be custom designed.

Team Composition and Technical Adaptability

It is now clear that the late awareness of the hardware demands revealed an inherent weakness within the structure of the team members. Since the project was initially viewed as a software

venture, there were too many students from Computer Engineering, as opposed to Electrical Engineering (five Computer Engineers compared to one Electrical Engineer). Therefore, it is evident that the team encountered significant difficulties in meeting the demand for the design of a circuit, as a result of the hardware demand.

Perseverance in Prototyping

Though there were certain issues with the scope of work and teamwork, the implementation process acted as a proof of adaptability within the team. Being forced to adapt quickly, the team had to think out of the box and look into the area beyond software and deal with problems with embedded systems and physical communications. Eventually, the team managed to create a functional prototype board that worked properly and transmitted NFC data in real time, proving their capability to build an effective system even under such conditions.

References

- [1] Monark, "Supply Chain Tracking Project Overview," Monark.io. [Online]. Available: <https://www.monark.io/en/project/supply-chain-tracking>.
- [2] Monark, "ChainProof Supply Chain Tracking Mockup Website," Monark.io. [Online]. Available: <https://chainproof.monark.io/>.
- [3] STMicroelectronics, "ST25DV16K: Dynamic NFC/RFID tag IC with 16-Kbit EEPROM, fast transfer mode and energy harvesting," Datasheet. [Online]. Available: <https://www.st.com/resource/en/datasheet/st25dv04k.pdf>
- [4] Espressif Systems, "ESP32-WROOM-32D / ESP32-DevKitC V4 Datasheet," Espressif. [Online]. Available: https://documentation.espressif.com/esp32-wroom-32_datasheet_en.pdf
- [5] Aosong Electronics, "DHT22 (AM2302) Digital Temperature and Humidity Sensor," Datasheet. [Online]. Available: <https://cdn-shop.adafruit.com/datasheets/DHT22.pdf>
- [6] OpenZeppelin, "OpenZeppelin Contracts: Secure Smart Contract Development," OpenZeppelin Documentation. [Online]. Available: <https://docs.openzeppelin.com/contracts/>
- [7] Vercel, "Next.js: The React Framework for the Web," Next.js Documentation. [Online]. Available: <https://nextjs.org/docs>
- [8] Ethereum Foundation, "Ethereum Smart Contract Documentation (Solidity)," Ethereum.org. [Online]. Available: <https://docs.soliditylang.org/>

Appendix A: The second ELSEE assignment.

Part A: 8-Factor ELSEE Considerations

8-Factor ELSEE Considerations	Primary Resource / Example	High Consideration	Moderate Consideration	Limited Consideration	Not Applicable	ELSEE Evidence [#]	Additional Notes / Explanation
1. Ethics – Code of Ethics (EC/OIQ)	Engineers Canada Code of Ethics	✓				[1][3]	Blockchain ensures integrity, transparency, and accountability. Environmental monitoring ensures product safety.
2. Ethics/Legal – OHS, WHMIS	Canadian OHS + WHMIS		✓			[2][20][21]	Hardware designed within safe temperature/power ranges; full industrial compliance not required for PoC.

3(a) Legal – Federal/Provincial Laws	Privacy & data laws (PIPEDA)		✓			[1][4]	Blockchain identity and traceability relate to data privacy and ownership laws.
3(b) Legal – Engineering Standards	EC/OIQ standards		✓			[2][19]	Secure architecture, RBAC, and cryptographic validation follow engineering best practices.
4(a) Manuals – Assembly	Hardware schematic			✓		[1][10]	Only PoC schematic provided; no full assembly manual required.
4(b) Manuals – Operations	UI flows & dashboards		✓			[1][12]	Role-based UI acts as operational guide for users.
4(c) Manuals – Troubleshooting	Testing methodology			✓		[1]	Testing described but no formal troubleshooting manual.
5. Legal – IP Compliance	Concordia IP Policy		✓			[30]	Work governed by Capstone/IP policies; no NDA explicitly stated.

6(a) Social Stakeholders –	Client Monark input	✓				[1][2]	Client feedback directly shaped system design and architecture decisions.
6(a) Indigenous Engagement	EC Indigenous guidelines				✓	[3]	No direct interaction with Indigenous communities.
6(b) JEDI Accessibility	Accessibility guidelines			✓		[29]	Basic usability considered; no formal accessibility compliance implemented.
7(a) Economic Budget –	Low-cost hardware		✓			[2][10][11]	Designed for affordability and scalability for small cooperatives.
7(b) Economic Components & Risks –	Hardware + alternatives analysis	✓				[2][10][11]	Component tradeoffs (ESP32, DHT22, BLE/NFC) analyzed with risks.
7(c) Economic Labour Impact –	Automation effects		✓			[22]	Improves efficiency but may reduce manual tracking roles.

7(d) Economic – Business Potential	Startup potential		✓			[22]	Scalable into real-world products or startup solutions.
8(a) Environment – Life Cycle	Sustainability guidelines			✓		[5]	No full lifecycle assessment conducted.
8(b) Environment – Energy/GHG	Blockchain energy usage		✓			[28]	Hybrid architecture reduces blockchain energy consumption.
8(c) Environment – Local Impacts	Supply chain waste reduction		✓			[26]	Reduces spoilage through environmental monitoring.
8(d) Environment – UN SDGs	UN SDGs	✓				[6]	Supports SDG 9, 12, and 8 (innovation, sustainability, economic growth).

PART B: Linked Cross-References From Matrix

[1] ChainProof Team, “ChainProof Final Report,” Concordia University, Montreal, QC, Canada, Apr. 2026.

[2] ChainProof Team, “ChainProof Phase 2 Design Report,” Concordia University, Montreal, QC, Canada, Nov. 2025.

[3] Engineers Canada, “Code of Ethics,” Ottawa, ON, Canada, 2023.

[4] Government of Canada, “Personal Information Protection and Electronic Documents Act (PIPEDA),” 2024.

- [5] Engineers Canada, “National Guideline on Sustainable Development and Environmental Stewardship,” 2022.
- [6] United Nations, “Sustainable Development Goals (SDGs),” 2023.
- [7] S. Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System,” 2008.
- [8] OpenZeppelin, “Smart Contract Security Guidelines,” 2024.
- [9] MetaMask, “MetaMask Documentation,” ConsenSys, 2025.
- [10] Espressif Systems, “ESP32 Technical Reference Manual,” 2023.
- [11] Aosong Electronics, “DHT22 Temperature and Humidity Sensor Datasheet,” 2022.
- [12] Vercel, “Next.js Documentation,” 2025.
- [13] PostgreSQL Global Development Group, “PostgreSQL Documentation,” 2025.
- [14] Solidity, “Solidity Programming Language Documentation,” Ethereum Foundation, 2025.
- [15] Tenderly, “Tenderly Platform Documentation,” 2025.
- [16] IEEE, “Blockchain for Supply Chain Traceability,” IEEE Trans. Eng. Manag., 2022.
- [17] International Organization for Standardization, “ISO/IEC 27001: Information Security Management Systems,” 2022.
- [18] International Organization for Standardization, “ISO 14001: Environmental Management Systems,” 2023.
- [19] Ordre des ingénieurs du Québec (OIQ), “Code of Ethics of Engineers,” Quebec, Canada, 2023.
- [20] Health Canada, “Workplace Hazardous Materials Information System (WHMIS),” 2023.
- [21] Government of Canada, “Canadian Occupational Health and Safety Regulations,” 2024.

- [22] Deloitte, “Supply Chain Traceability and Transparency Report,” 2023.
- [23] National Institute of Standards and Technology (NIST), “IoT Security Best Practices,” 2023.
- [24] ConsenSys, “Web3 Security Best Practices,” 2024.
- [25] ACM, “Data Integrity in Distributed Systems,” Commun. ACM, 2021.
- [26] IEEE, “Environmental Monitoring Systems Using IoT Sensors,” 2022.
- [27] Trail of Bits, “Smart Contract Security and Audit Guidelines,” 2023.
- [28] Cambridge Centre for Alternative Finance, “Global Cryptoasset Benchmarking Study (Energy Use),” 2023.
- [29] W3C, “Web Content Accessibility Guidelines (WCAG) 2.1,” 2018.
- [30] Concordia University, “Intellectual Property Policy (VPRGS-9),” Montreal, QC, Canada, 2024.

PART C: Stakeholders

Stakeholder	Affiliation / Title / Sector	Proposed Follow-Up / Contact
Monark (Client)	Loucas Pelletier (COO), Dominic Fournier (CTO), Blockchain Supply Chain Industry	Continue collaboration for real-world deployment and feedback
Capstone Supervisor (Dr. Yan Liu)	Concordia University,, Professor in the Electrical and Computer Engineering Department	Review design decisions and provide technical validation

Producers (Coffee Suppliers)	Agricultural Sector	Gather feedback on usability and traceability needs
Transporters / Logistics Operators	Supply Chain / Transportation	Evaluate real-world integration and data input workflows
Consumers	End Users	Assess transparency, trust, and interface usability

PART D: Consideration of Risk/Failure & Remedies in Our Capstone Domain

A key technological risk in blockchain-based supply chains is the integrity of IoT sensor data, as incorrect or manipulated environmental readings can compromise the reliability of the system. In ChainProof, this risk is mitigated by combining hardware sensing with cryptographic hashing and blockchain anchoring to ensure data immutability. However, failures such as sensor malfunction, communication loss, or incorrect calibration may still occur. Future designs could address these risks through redundancy (multiple sensors), periodic calibration, and validation mechanisms in the backend. These measures would improve system robustness and ensure trustworthy traceability.

PART E: Possible Venues for Publication of Our Findings

Possible venues for publishing this work include IEEE conferences on blockchain and IoT systems, such as IEEE ICC or IEEE Blockchain Conference, as well as journals focused on supply chain technologies. Industry venues such as ACM conferences or Web3 developer forums may also be suitable for presenting applied research and prototype implementations.

Appendix B: Technical Manual

1. Introduction

This technical manual describes the design and operation of the ChainProof prototype from an engineering perspective. The manual focuses on how the prototype is built, how its subsystems interact, how data moves from the physical device into the web platform and blockchain layer, and how the system can be tested and debugged.

The current ChainProof implementation is a proof-of-concept for decentralized supply-chain tracking. It combines a tethered IoT sensing device, NFC-based interaction, a web platform, wallet authentication, and Ethereum-compatible smart contracts. The prototype demonstrates how a physical batch can be associated with a digital identity, verified by scan, and processed through role-controlled custody actions.

2. Objective

The objective of the ChainProof prototype is to provide a technical foundation for traceable batch handling in a supply-chain environment. The prototype is specifically designed to:

- collect environmental context from a physical device,
- expose that context through NFC deep linking,
- identify the corresponding batch through a hardware identifier,
- enforce role-based custody actions on-chain,
- provide a browser-based interface for producers, warehouses, and transporters.

From a technical standpoint, the project demonstrates integration across embedded firmware, NFC memory, backend verification, a web application, wallet-based signing, and a smart-contract state machine.

3. Product Specifications

The table below summarizes the current prototype-level technical specifications.

Parameter	Implemented value / behavior	Notes
Processing platform	ESP32 development board class device	Firmware implemented in <code>services/iot/blink/main/main.c</code>
Environmental sensor	DHT22	Reads temperature and humidity
NFC device	ST25DV16K	Used for NFC-readable memory and I2C updates
Firmware sample period	3000 ms	Prototype firmware constant <code>SAMPLE_PERIOD_MS</code>
NFC URI refresh minimum interval	60000 ms	Limits how often the NFC deep link is rewritten
Telemetry mirror refresh minimum interval	30000 ms	Used for mirrored NFC telemetry region updates
Temperature threshold window	-5.0 C to 27.0 C	Outside this range sets temperature out-of-range flag
Humidity threshold window	-5.0 % to 40.0 %	Outside this range sets humidity out-of-range flag
NFC payload version	2	Includes monotonic counters and signature
Hardware identifier format	Uppercase hex string derived from device MAC	Used as the hardware-to-batch lookup key
Signature method	HMAC-SHA256, truncated to 128 bits	Encoded in the sig field of the URI payload

Role model in deployed UI	Producer, Warehouse, Transporter	Current user-facing roles
Contract platform	Ethereum-compatible smart contract	ChainProof.sol
Batch mass unit	Weight in kilograms	Exposed in the current web UI as Weight (kg)
Web platform	Next.js web application	Provides dashboards, tracking, and action pages
Wallet model	MetaMask / WalletConnect compatible	Transactions remain user-approved in wallet

4. System Block Diagram

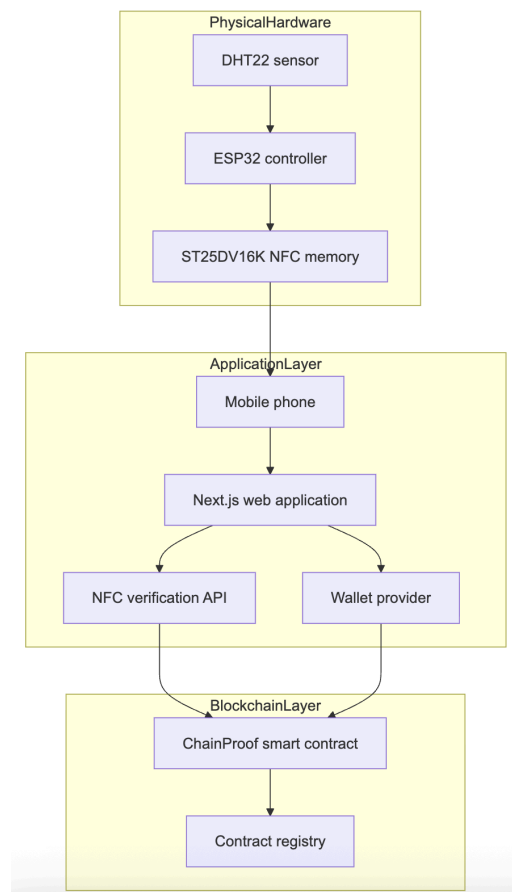


Figure 5: High-level technical block diagram of the ChainProof system

5. Description of Operation

The prototype operates as a cross-layer system. The DHT22 sensor periodically provides temperature and humidity readings to the firmware running on the ESP32 controller. The firmware maintains counters, applies threshold logic, generates a signed NFC payload, and writes the resulting deep link to the ST25DV16K memory over I2C. When a user taps the NFC-enabled hardware with a smartphone, the phone opens the ChainProof web application using the deep link stored on the tag.

The web application receives the query payload and forwards it to a verification endpoint. After successful verification, the application queries the blockchain to resolve the `hardware_id` to a batch. The platform then checks the connected wallet and the role assigned to that wallet. Based on that role, the system exposes only the actions allowed for the current actor. Any state-changing action is finalized only when the user approves the corresponding blockchain transaction in the wallet.

This design separates sensing, verification, interface, and state enforcement into distinct layers while preserving an end-to-end path from physical interaction to immutable digital record.

6. Hardware description

6.1 Main Hardware Components

The implemented system uses a compact set of hardware components:

Component	Function in the prototype
ESP32 development board	Main embedded controller for sampling, processing, signing, and I2C communication
DHT22	Measures ambient temperature and humidity
ST25DV16K	Stores the NFC deep link and mirrored telemetry content for smartphone access
USB power connection	Powers the tethered prototype during demonstration and testing
Protoboard / soldered assembly	Hosts the proof-of-concept interconnections between modules

6.2 Hardware Interconnection Summary

The firmware constants define the prototype wiring used by the current implementation.

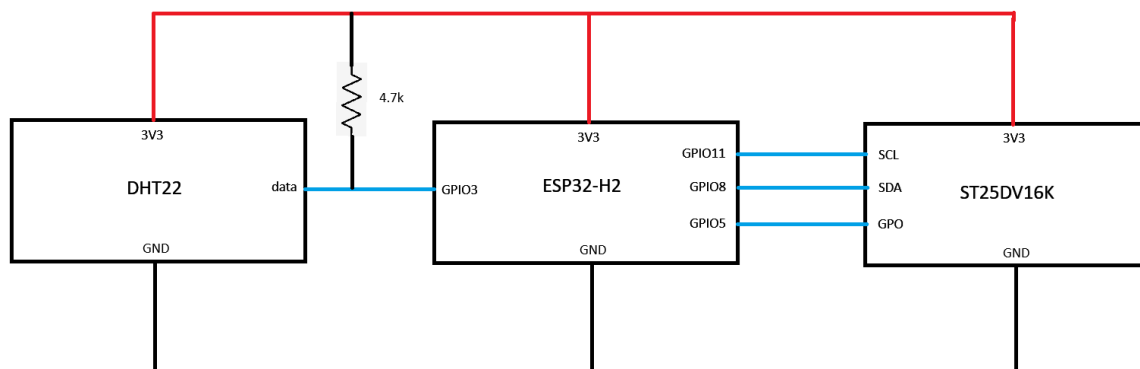


Figure 6: Hardware Schematic

Signal	Firmware constant	GPIO / interface
DHT22 data	DHT_GPIO	GPIO_NUM_3
ST25DV SDA	ST25DV_I2C_SDA_GPIO	GPIO_NUM_8
ST25DV SCL	ST25DV_I2C_SCL_GPIO	GPIO_NUM_11

ST25DV GPO	ST25DV_GPO_GPIO	GPIO_NUM_5
ST25DV I2C port	ST25DV_I2C_PORT	I2C_NUM_0
ST25DV user address	ST25DV_I2C_USER_ADDR_7BIT	0x53

6.3 Hardware Functional Description

- The ESP32 controller initializes the DHT22, NFC state storage, I2C bus, and ST25DV interface. It also derives a unique `hardware_id` from the device MAC address.
- The DHT22 sensor provides the environmental measurements used to determine the current telemetry state and threshold flags.
- The ST25DV16K provides dual access to the same stored information: the microcontroller updates it through I2C, while the user accesses it through NFC.
- The GPO line supports ST25DV signaling and helps with tag interaction management in the embedded workflow.

7. Software and Firmware Description

7.1 Firmware Description

The system software runs on the ESP32-H2 and controls the overall operation of the device. The program periodically wakes the microcontroller every 30 minutes to read temperature and humidity data from the DHT22. The measured values are then processed to update the minimum and maximum temperature and humidity records. After processing, the updated data is written to the ST25DV16K through the I2C interface. This allows the recorded information to be stored in NFC memory. When a user taps the device with a smartphone, the phone can read the stored data

via NFC and upload it to the Internet or a monitoring application. To reduce power consumption, the software places the microcontroller in sleep mode between measurement intervals, allowing the device to operate efficiently for long-term environmental monitoring. It performs the following functions:

1. initialize non-volatile state used for boot and sequence counters,
2. derive a hardware identifier from the ESP32 MAC address,
3. periodically read temperature and humidity from the DHT22,
4. compare readings against configured thresholds,
5. generate a canonical payload string,
6. compute a truncated HMAC-SHA256 signature,
7. publish the resulting deep link to the ST25DV16K.

The firmware also maintains monotonic counters such as `boot_id`, `nfc_seq`, and `sample_seq`. These counters help the platform detect repeated or replayed scan payloads.

7.2 Firmware Flow Chart

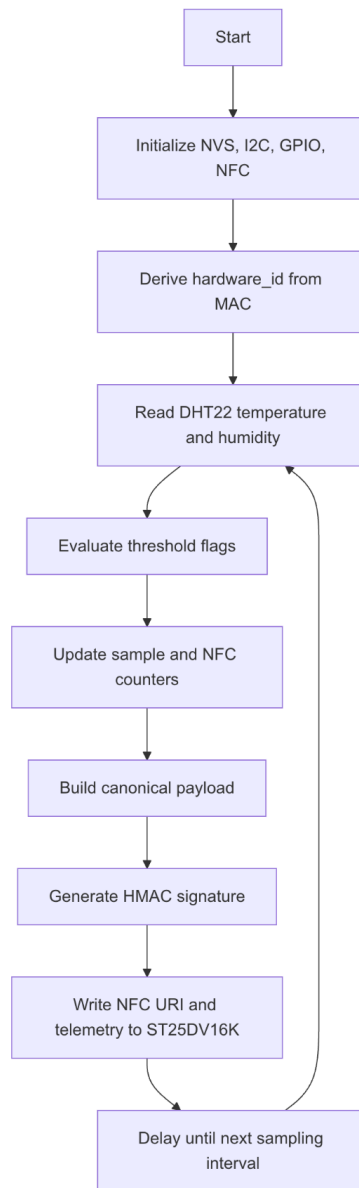


Figure 7: Firmware control flow for sensing, payload generation, and NFC publication.

7.3 Web Application Description

The web application is implemented in services/web using Next.js. Its main functions are:

- parse the NFC deep-link query parameters,
- call the verification endpoint,
- resolve the batch by `hardware_id`,
- manage wallet sessions and role-aware routing,
- present dashboards and the Batch Actions page,
- submit blockchain transactions for batch creation, transfer initiation, and receipt.

The web layer is intentionally not the final source of truth for permissions. It is responsible for orchestration, validation, and presentation, while the contract remains the authority on allowed actions.

7.4 Wallet Authentication Description

Wallet authentication is handled by `wallet-auth-context.tsx` and related helpers in the web service. The platform:

- connects to wallet providers,
- reconciles the current wallet chain with the configured ChainProof network,
- reads the on-chain role for the connected address,
- stores the runtime session needed for transaction signing.

Supported user-facing wallet modes include MetaMask and WalletConnect, with optional debug-only manual wallet fallback in local development contexts.

7.5 Smart Contract Description

The ChainProof.sol contract implements the state machine for custody and batch recognition.

The main contract responsibilities are:

- store actor roles,
- create and store batch records,
- bind and resolve hardware identifiers,
- enforce transfer and receipt logic,
- preserve parent-child lineage for split-based flows,
- emit events for important lifecycle transitions.

In the current implementation, the main operational functions exposed in the user-facing workflows are:

- harvestBatch
- harvestBatchWithHardware
- bindHardwareIdToBatch
- initiateTransfer
- receiveBatch
- splitBatch

7.6 Role-Specific Logic

- Producer creates new batches and binds hardware.
- Warehouse receives custody and can perform warehouse-side handling of batches.

- Transporter receives shipments and transfers them onward.

The contract validates that the sender has the correct role and, where required, is also the current handler of the batch.

8. Test and Debugging

The system was tested in a laboratory prototype environment to verify both hardware and software functionality. The prototype consisted of an ESP32-H2, a DHT22, and an ST25DV16K connected on a prototype board and powered through a USB connection. Initial debugging focused on verifying stable power supply, correct sensor readings, and proper communication between components. During testing, the microcontroller periodically woke from sleep mode and read temperature and humidity values from the DHT22 sensor. The readings were checked for accuracy and consistency by comparing them with known environmental conditions. The software was also verified to correctly calculate and update the minimum and maximum values measured over time. These values were then written to the ST25DV16K through the I2C interface. NFC communication testing was performed by tapping a smartphone on the NFC antenna connected to the ST25DV16K. The stored environmental data was successfully retrieved through NFC, confirming correct memory access and tag functionality. Additional debugging included verifying loC communication signals, ensuring pull-up resistors were functioning correctly, and confirming that the device entered sleep mode between measurement intervals to reduce power consumption. System integration testing confirmed that sensor data could be reliably captured, stored, and retrieved through NFC without data loss. The prototype demonstrated stable operation throughout the testing period, indicating that the hardware connections, firmware logic, and NFC data transfer mechanisms were functioning correctly.

9. Technical Limitations and Recommended Improvements

The current prototype is effective as a proof of concept, but several technical limitations remain:

- it is tethered rather than battery-powered,
- it uses a development-board style implementation rather than a finalized PCB,
- the prototype emphasizes scan-driven integration over long-term autonomous deployment.

Recommended future improvements include:

- designing a dedicated PCB and enclosure,
- adding production-grade power management and battery support,
- expanding the hardware qualification and environmental validation process,
- formalizing a supplier-grade BOM and annotated schematic appendix.

Appendix C: User Manual

Purpose

This manual explains how to use the ChainProof platform as an operator. It is written for users who interact with the deployed website to scan NFC-enabled hardware, connect a wallet, and complete batch actions as a producer, warehouse operator, or transporter.

Who Should Use This Manual

- Producer users who create batches, bind hardware, and initiate the first transfer.
- Warehouse users who receive batches and transfer them onward.
- Transporter users who receive shipments and deliver them to the next approved actor.

Before You Start

Before using the platform, make sure the following are available:

Requirement	What you need
Website access	Access to the deployed ChainProof website
Wallet	A supported wallet such as MetaMask or WalletConnect
Correct network	Your wallet must be connected to the ChainProof blockchain network configured for the deployment
Assigned role	Your wallet must be assigned an on-chain role: Producer, Warehouse, or Transporter
NFC device	A physical NFC-enabled device attached to the batch you want to work with

Important Usage Notes

Most operational actions begin with an NFC scan. Scanning loads the batch context needed by the Batch Actions page. If you open the Batch Actions page without first scanning a tag, required

batch context may be missing. The platform shows only the actions allowed for your currently assigned role. Transaction approval always happens in your connected wallet. A button click in the web app does not complete the action until the wallet transaction is approved.

Main Pages

Page	Purpose
Dashboard	Shows the main view for your current role
Batches	Lets you track batches by batch ID or tracking code
Batch Actions	Handles NFC-based actions such as harvest, receive, and transfer
Sign In	Connects your wallet
Assign Role	Assigns an on-chain role if your wallet does not already have one

Sign In and Connect Your Wallet

1. Open the ChainProof website.
2. Select Sign In if you are not already connected.
3. Choose your wallet connection method.
4. Approve the wallet connection request.
5. If your wallet asks to switch or add the ChainProof network, approve that request.
6. Wait for the platform to read your connected wallet and resolve your role.

Apple Device Note

On iPhone or iPad, use WalletConnect and approve the request in the MetaMask app or your chosen wallet app.

Assign Your Role

If your wallet has no assigned role, the platform sends you to the Assign Role page.

1. Confirm that the connected wallet address is correct.
2. Open the role selector.
3. Choose one of the available roles:
 - Producer
 - Warehouse
 - Transporter
4. Select Assign Role On-Chain.
5. Approve the role assignment transaction in your wallet.
6. Wait for the platform to return you to the correct page.

Start an NFC-Based Workflow

1. Use your phone to scan the NFC-enabled hardware attached to the batch.
2. Allow the scan to open the ChainProof website.
3. Wait while the platform:
 - parses the signed payload,
 - verifies the scan with the backend,

- looks up the batch using the hardware identifier,
 - checks your wallet and role.
- 4. If your wallet is not connected, you will be sent to Sign In.
- 5. If your wallet has no role, you will be sent to Assign Role.
- 6. If the scan succeeds, the platform opens Batch Actions with the batch context loaded.

Producer Workflow

Use this workflow when a hardware device has not yet been mapped to a batch, or when you need to create a new batch and bind it to that hardware.

Create a Batch and Bind Hardware

1. Scan the hardware tag.
2. On the Batch Actions page, confirm that the hardware ID is shown.
3. Enter the batch details:
 - Origin / Product Label
 - Tracking Code
 - Weight (kg)
4. Confirm that the hardware ID field is populated.
5. Select Harvest & Bind Hardware.
6. Approve the transaction in your wallet.
7. Wait for the success message confirming that the batch was created and the hardware mapping was updated.

Transfer the Batch

After the batch has been created and mapped:

1. Confirm that the Resolved batch value is visible.
2. Enter the recipient wallet address.
3. Select Initiate Transfer.
4. Approve the transaction in your wallet.
5. Wait for confirmation that the transfer was initiated.

Warehouse Workflow

Use this workflow when the warehouse is receiving or forwarding custody of an already mapped batch.

Receive a Batch

1. Scan the hardware tag for the incoming batch.
2. Confirm that the platform resolves the correct batch.
3. Select Receive Batch.
4. Approve the transaction in your wallet.
5. Wait for confirmation that the batch is now in warehouse custody.

Transfer a Batch Onward

1. Scan the hardware tag for the batch currently in warehouse custody.
2. Confirm that the resolved batch is correct.

3. Enter the recipient wallet address.
4. Select Initiate Transfer.
5. Approve the transaction in your wallet.
6. Wait for confirmation that the transfer was initiated.

Transporter Workflow

Use this workflow when the transporter is taking custody of a shipment and then handing it to the next approved actor.

Receive a Shipment

1. Scan the hardware tag for the shipment.
2. Confirm that the batch is resolved.
3. Select Receive Shipment.
4. Approve the transaction in your wallet.
5. Wait for confirmation that the shipment is now in transporter custody.

Deliver a Shipment

1. Scan the hardware tag for the shipment currently in your custody.
2. Enter the destination wallet address.
3. Select Deliver Shipment.
4. Approve the transaction in your wallet.
5. Wait for confirmation that the transfer was initiated.

Track a Batch

Use the Batches page when you want to look up a batch and keep it in your tracked list.

1. Open Batches.
2. Select Track Batch.
3. Enter either:
 - a numeric batch ID, or
 - a tracking code.
4. Submit the lookup.
5. Review the returned batch details.
6. Open View Details to inspect status, custodian, timestamps, parents, children, and batch history.

Note About Tracked Batches

The tracked list is saved in your browser for the connected wallet. It is not a global list of every batch on-chain.

Common Problems and What To Do

Wrong Network

If the platform says your wallet is on the wrong chain:

1. Reopen Sign In.

2. Reconnect your wallet.
3. Approve the network switch request when prompted.

No Role Assigned

If the platform shows Role not assigned:

1. Open Assign Role.
2. Choose your correct role.
3. Approve the on-chain role assignment transaction.

No Batch Found for Hardware

If the hardware is scanned but no batch is resolved:

- The hardware has not been mapped yet, or
- the mapping is not available on the current deployment.

If you are a producer, create and bind the batch. If you are a warehouse or transporter, contact the producer or administrator.

No Scan Context Available

If the Batch Actions page says that no NFC context is available:

1. Return to the physical hardware.
2. Scan the NFC tag again.
3. Let the scan reopen the site.

Wallet Transaction Rejected

If a transaction fails because it was rejected:

- reopen the action,
- confirm the details,
- approve the request in your wallet.

Recipient Address Problems

If a transfer fails:

- check that the recipient wallet address is valid,
- make sure the recipient already has an assigned role,
- confirm that the transfer route is allowed by the workflow.

What the Platform Verifies

Type of check	Where it happens	Purpose
NFC payload verification	Backend verification service	Confirms that the scanned payload is authentic and not stale or replayed
Batch lookup	Smart contract read	Resolves the hardware identifier to the correct batch

Role permissions	Smart contract	Ensures only allowed actors can perform the requested action
State updates	Smart contract transaction	Stores the authoritative custody result on-chain

Good Operating Practice

- Always confirm the hardware ID and resolved batch before submitting an action.
- Do not assume a click in the web app is complete until the wallet transaction is approved.
- Rescan the tag if the action page loses its batch context.
- Use the Batches page to review history after important actions.
- Disconnect your wallet when you finish working on a shared device.